



Programmation Orientée Objets

(II 1)

L'héritage en C++



L'héritage en C++

- Concept & Principe
- Visibilité entre héritier
- Redéfinition des méthodes
- Constructeurs & destructeurs
- Type d'héritage
- Héritage multiple

1. Concept & Principe

- *Étendre un **type abstrait** :*
 - *maj. des services et/ou des comportements*
- *Mécanisme permettant de :*
 - *ajouter de nouvelles fonctionnalités à une classe existante*
 - *changer un peu le comportement de certaines méthodes d'une classe déjà existante*

On veut faire cela sans rien changer à la classe déjà existante

↩ *On définira donc une nouvelle classe qui héritera de la classe existante*

- *En C++, on parlera de **classe de base** et de **classe dérivée***

(suite)

- Une classe dérivée **hérite** les attributs et les méthodes de la classe de base (à l'exception de quelques unes, e.g les méthodes amies)
- Mais, une classe dérivée peut **redéfinir** une méthode de la classe de base.
 - C'est cette méthode redéfinie **qui sera appelée pour un objet de cette classe**, et non pas la méthode originale de la classe supérieure
 - Elle **masquera** la méthode de la classe de base

Exemple : classe de base

■ Voyons d'abord la classe Employé:

```
class Employe {  
    public:  
        Employe(std::string nom_employe, double salaire_initial);  
        void setSalaire(double nouveau_salaire);  
        double getSalaire() const;  
        std::string getNom() const;  
    private:  
        string nom;  
        double salaire;  
};
```

Exemple : classe dérivée

```
class Directeur : public Employe {  
    public:  
        Directeur(std::string nom_employe, double salaire_initial);  
        void ajouterEmploye(Employe* employe);  
        Employe getEmploye(std::string& nom) const;  
    private:  
        std::vector<Employe*> employes_supervises; //Collection d'employés  
};
```

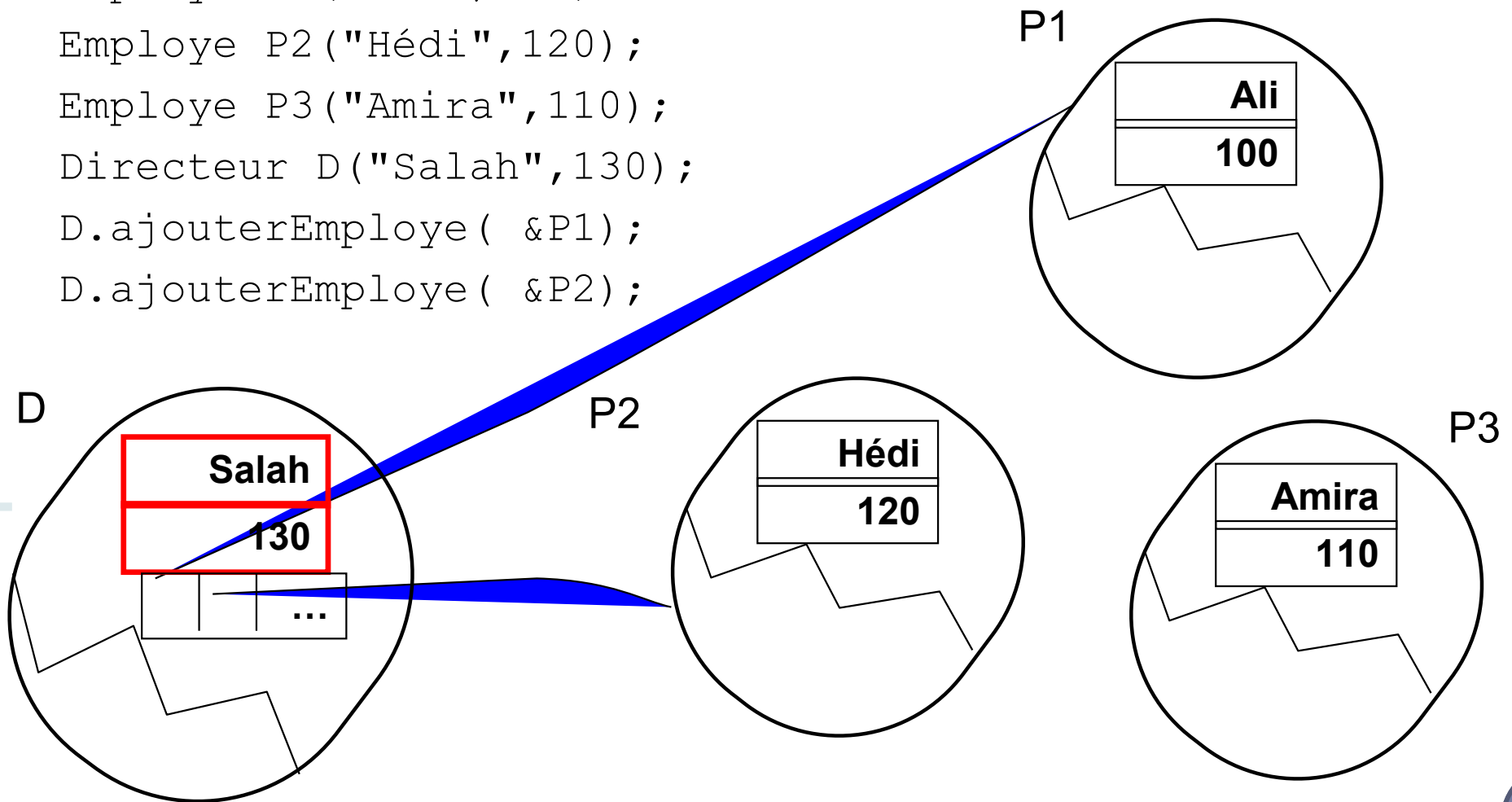
héritage

Attribut spécifique à l'héritier

Nouvelle méthode

Instanciation d'une classe dérivée

```
Employe P1("Ali",100);  
Employe P2("Hédi",120);  
Employe P3("Amira",110);  
Directeur D("Salah",130);  
D.ajouterEmploye( &P1);  
D.ajouterEmploye( &P2);
```



Remarques

- L'héritage est une relation de type « **est un** », « **is a** »
 - Directeur est une classe dérivée de la classe Employe.
 - ➔ Tout directeur est un employé, ce qui est tout à fait conforme à l'intuition
- Il ne faut pas confondre l'héritage avec l'agrégation (ou la composition)
 - Il pourrait être tentant d'utiliser l'agrégation(ou la composition) au lieu de l'héritage
 - Du point de vue technique, le résultat peut paraître équivalent. Mais il s'agit de deux concepts tout à fait différents
 - Héritage = spécialisation $\neq$ Composition=utilisation

Exemple 2: un point coloré est un point

```
class point
```

```
{
```

```
/* déclaration des membres privés */
```

```
int x ;
```

```
int y ;
```

```
/* déclaration des membres publics */
```

```
public :
```

```
void initialise (int, int) ;
```

```
void deplace (int, int) ;
```

```
void affiche () ;
```

```
} ;
```

```
class pointcol : public point
```

```
// pointcol dérive de point
```

```
{
```

```
    short couleur ;
```

```
    public :
```

```
        void colore (short cl)
```

```
            { couleur = cl ; }
```

```
};
```

Exemple 2: un point coloré est un point

```
/* --- Déclaration et définition de la classe pointcol ----- */
```

```
class pointcol : public point // pointcol dérive  
    de point
```

```
{ short couleur ;
```

```
public :
```

```
void colore (short cl) { couleur = cl  
};
```

```
main()
```

```
{ pointcol p ;
```

```
p.initialise (10,20) ; p.colore (5) ;
```

```
p.affiche () ;
```

```
p.deplace (2,4) ;
```

```
p.affiche () ;
```

```
}
```

Chaque objet de type pointcol peut alors faire appel :

- aux méthodes publiques de pointcol (ici colore),
- aux méthodes publiques de la classe de base point (ici initialise, deplace et affiche).

Je suis en 10 20
Je suis en 12 24

Type d'héritage

- Il existe trois différents types d'héritage, soient : ***public***, ***private*** et ***protected***.
- Le type d'héritage est spécifié après le symbole :
`class Clock : public Horloge`
- **Par défaut**, le type d'héritage est **privé**.

`class Clock : Horloge` ⇔ `class Clock : private Horloge`

Le Type d'héritage **conditionne**
la visibilité des héritiers

Héritage public

```
class ClasseDerivee : public ClasseBase
```

Classe de base		Classe dérivée
<code>private</code>	→	Inaccessible
<code>protected</code>	→	<code>protected</code>
<code>public</code>	→	<code>public</code>

Les attributs *private*, *protected* et *public* de la classe de base restent les mêmes pour la classe dérivée.

Exemple : visibilité

```
class A {  
    private :  
        int i;  
    public :  
        int j;  
    protected :  
        int k;  
};
```

```
void main()  
{  
    A a;  
    a.i = 0;  
    a.j = 0;  
    a.k = 0;  
}
```

```
class B : public A {  
    public :  
        void f()  
        {  
            i = 0; //error  
            j = 0;  
            k = 0;  
        }  
};
```

```
void main()  
{  
    ...  
    B b;  
    b.i = 0;  
    b.j = 0;  
    b.k = 0;  
}
```

Héritage protected

```
class ClasseDerivee : protected ClasseBase
```

Classe de base		Classe dérivée
<code>private</code>	→	Inaccessible
<code>protected</code>	→	<code>protected</code>
<code>public</code>	→	<code>protected</code>

Les attributs ***public*** de la classe de base deviennent protégés pour la classe dérivée.

Exemple : visibilité

```
class A {  
    private :  
        int i;  
    public :  
        int j;  
    protected :  
        int k;  
};
```

```
void main()  
{  
    A a;  
    a.i = 0;  
    a.j = 0;  
    a.k = 0;  
}
```

```
class B : protected A {  
    public :  
        void f()  
        {  
            i = 0; //error  
            j = 0;  
            k = 0;  
        }  
};
```

```
void main()  
{  
    ...  
    B b;  
    b.i = 0;  
    b.j = 0;  
    b.k = 0;  
}
```

Héritage privé

```
class ClasseDerivee : private ClasseBase
```

Classe de base		Classe dérivée
<code>private</code>	→	Inaccessible
<code>protected</code>	→	<code>private</code>
<code>public</code>	→	<code>private</code>

Tous les attributs de la classe de base deviennent *private* pour la classe dérivée.

Exemple : visibilité

```
class A {  
    private :  
        int i;  
    public :  
        int j;  
    protected :  
        int k;  
};
```

```
void main()  
{  
    A a;  
    a.i = 0;  
    a.j = 0;  
    a.k = 0;  
}
```

```
class B : private A {  
    public :  
        void f()  
        {  
            i = 0; //error  
            j = 0;  
            k = 0;  
        }  
};
```

```
void main()  
{  
    ...  
    B b;  
    b.i = 0;  
    b.j = 0;  
    b.k = 0;  
}
```

Droits d'accès sur les membres hérités

		mot clé utilisé pour l'héritage		
Accès aux données		public	protected	private
mot clé utilisé pour les champs et les méthodes	public	public	protected	private
	protected	protected	protected	private
	private	inaccessible	inaccessible	inaccessible

2. Visibilité entre héritier

- Lors de l'héritage public, les attributs privés de la classe de base deviennent inaccessibles pour les classes dérivées.
 - ↳ Il faut donc utiliser les **méthodes publiques** de la classe de base afin d'accéder à ces attributs
- Les attributs protégés de la classe de base seront accessibles par les classes dérivées, mais ne seront pas accessibles par les clients de la classe dérivée.

Utilisation des membres de la classe de base dans une classe dérivée

```
class pointcol : public point
{ ...
void affichec ()
{ cout << "Je suis en " << x <<
  " " << y << "\n" ;
cout << " et ma couleur est : "
  << couleur << "\n" ;
}
}
```

```
void pointcol::affichec ()
{
  affiche () ;
  cout << " et ma couleur est : " <<
    couleur << "\n" ;
}
```

```
void pointcol::initialisec (int
abs, int ord, short cl)
{ initialise (abs, ord) ;
  couleur = cl ;
}
```

➤ Une méthode d'une classe dérivée n'a pas accès aux membres privés de sa classe de base.

Exercice d'application (Ex1/TD2)

Soient les deux classes C++ suivantes et la fonction main qui permet de les utiliser, indiquer les erreurs et proposer des corrections:

```
class Rectangle
{
protected :

void changer_largeur(int);
void changer_longueur(int);

public:

int largeur, longueur;

Rectangle(int l=5, int L=10) ;
int surface() const;
int Perimetre();

};
```

```
class Carre : private Rectangle
{
public :

Carre(int cote=10);
void changer_cote(int);

};
```

```
int main()
{
Rectangle R;

Carre c1, c2(13,14);
R.longueur =10;

R.largeur = 11;

c1.longueur =22;

c1.largeur = 22;

cout << c1.surface();
R.changer_cote(32);
cout <<c1.Perimetre();
cout <<R.Perimetre();

}
```

Exemple : Horloge

- Soit la classe Horloge, qui permet d'obtenir l'heure locale, de deux façons:

➡ normale : **(23:45)**

➡ à l'américaine [am/pm] **(pm 11:45)**

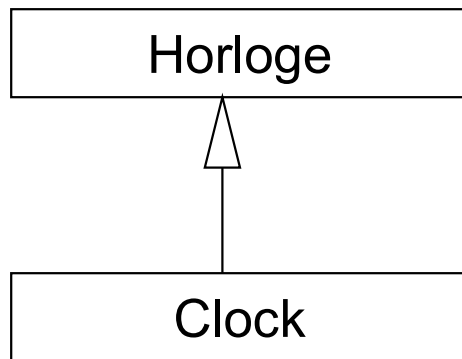
Cas : Horloge sans héritage

```
class Horloge {  
    CC hh,mm;  
    bool a_americaine;  
public :  
    Horloge(int h, int m,  
            bool americaine = false)  
        :hh(h,24),mm(m,60)  
        {a_americaine = americaine;}  
    void operator ++();  
    int valh();  
    int valm();  
    void afficher();  
};
```

```
void Horloge::afficher()  
{  
    if ( a_americaine)  
        // am h:m ou pm h:m  
    else  
        // h:m  
}
```

```
void main()  
{  
    Horloge H1(22,50,true);  
    Horloge H2(23,55,false);  
}
```

Cas : Horloge avec héritage



```
class Clock : public Horloge
{
    public :
        Clock(int h, int m);
        ...
};
```

```
class Horloge {
    CC hh,mm;
public :
    Horloge(int h, int m);
    int valh();
    int valm();
    void afficher();
};
```

```
void main()
{
    Horloge H1(22,50);
    Clock H2(23,55);
}
```


3. Redéfinition des méthodes de base

- Les méthodes de la classe de base peuvent être **redéfinies** dans la classe dérivée.
- Les méthodes redéfinies de la classe de base demeurent accessibles via l'opérateur de résolution de portée ("::").

Cas : Clock – classe dérivée

```
class Clock : public Horloge
{
public :
    Clock(int h, int m);
    void afficher ();
};
```

```
void Clock::afficher()
{
    if ( valh() < 12 )
        // am h:m
    else
        // pm h-12:m
}
```

```
void main()
{
    Clock H1(22,50);
    H1.afficher();
    H1.Horloge::afficher();
}
```

Exemple : Redéfinition de méthodes

```
void Horloge::afficher()
{
    cout<< valh() << ":" << valm() << endl;
}
```

```
void Clock::afficher()
{
    if ( valh() < 12 ) {
        cout << "am ";
        Horloge::afficher();
    } else {
        cout << "pm " << valh() - 12 << ":" << valm() << endl;
    }
}
```

**Appel de la fonction
afficher() du parent**



Redéfinition des fonctions membres d'une classe dérivée

```
class pointcol : public  
point{  
public :  
    void colore (short cl)  
    { couleur = cl ; }  
    // redéfinition de affiche de point  
    void affiche () ;  
    void initialise (int, int, short);  
};
```

```
void pointcol::affiche ()  
{  
    point::affiche () ; // appel de affiche de la classe  
    point  
    cout << couleur << "\n" ;  
}  
  
void pointcol::initialise (int abs, int ord,  
short cl)  
{  
    point::initialise (abs, ord) ;  
    couleur = cl ;  
}
```

```
void main(){  
    pointcol p ;  
    p.initialise (10,20, 5) ; p.affiche () ;  
    p.point::affiche () ; // pour forcer l'appel de affiche de point  
    p.deplace (2,4) ; p.affiche () ;  
    p.colore (2) ; p.affiche () ; }  
}
```

Redéfinition des membres données d'une classe dérivée

```
class A
{ .....
int a ;
char c ;
.....
};
```

```
class B : public A
{ float a ;
.....
};
```

NB. le membre a défini dans B s'ajoute au membre a hérité de A ;
il ne le remplace pas.

```
main()
{ B b;
b.a; // fait référence au membre a de type float de b.
b.A::a; // fait référence au membre a de type int de a.
};
```

Redéfinition et surdéfinition

```
class A
```

```
{ public :
```

```
void f(int n) { ..... }
```

```
void f(char c) { ..... }
```

```
// f est surdéfinie dans A
```

```
};
```

```
class B : public A
```

```
{ public :
```

```
void f(float x) { ..... }
```

```
// on ajoute une troisième  
définition dans B
```

```
};
```

lorsqu'une fonction est redéfinie dans une classe dérivée, elle **masque** une fonction de même signature de la classe de base.

```
main()
```

```
{ int n ; char c ; A a ; B b ;
```

```
a.f(n) ; // appelle A::f(int) (règles habituelles)
```

```
a.f(c) ; // appelle A::f(char) (règles habituelles)
```

```
b.f(n) ; // appelle B::f(float) (alors que peut-être A::f(int) conviendrait)
```

```
b.f(c) ; // appelle B::f(float) (alors que peut-être A::f(char) conviendrait)
```

```
}
```

4. Méthodes non héritées

Les classes dérivées n'héritent pas :

- Des constructeurs (défaut, paramètres, copie);
- Du destructeur;
- De l'opérateur d'affectation;
- Des relations d'amitié.

5. Constructeur et destructeur

- Lors de la création d'un objet d'une classe dérivée, les constructeurs sont appelés dans l'ordre suivant:
 - Les constructeurs des objets attributs de la classe de base,
 - Le constructeur de la classe de base,
 - Les constructeurs des objets attributs de la classe dérivée;
 - Le constructeur de la classe dérivée.
- Les destructeurs sont appelés en ordre inverse des constructeurs.

Exemple : constructeur

```
class A {  
    public :  
    A() { cout << "A::A()" << endl;}  
    ~A() { cout << "A::~~A()" << endl;}  
};
```

```
void main()  
{  
    B b;  
    cout << "***" << endl;  
}
```

```
class B : public A {  
    public :  
    B() { cout << "B::B()" << endl;}  
    ~B() { cout << "B::~~B()" << endl;}  
};
```

```
A::A()  
B::B()  
**  
B::~~B()  
A::~~A()
```

Transmission d'informations entre constructeurs

```
class point  
{ ...  
public :  
point (int , int );  
...  
~point ();  
...  
};
```

```
class pointcol : public point  
{ ...  
public :  
pointcol (int, int, short) ;  
~pointcol ();  
...  
};  
...  
pointcol::pointcol (int abs, int ord,  
    short cl) : point (abs, ord)  
{  
...  
}
```

pointcol retransmet à *point*
les deux premières
informations reçues

Constructeur de Horloge & Clock

```
Horloge::Horloge(int h, int m)
: hh(h) , mm(m)
{
}
}
```

```
Clock::Clock(int h, int s) : Horloge(h,s)
{
}
}
```

CAS DU CONSTRUCTEUR DE RECOPIE

- Rappelons que le constructeur de copie (qu'il s'agisse de ce lui par défaut ou de ce lui qui est fourni explicitement) est appelé en cas :
 - d'initialisation d'un objet par un objet de même type,
 - de transmission de la valeur d'un objet en argument ou en retour d'une fonction

CAS DU CONSTRUCTEUR DE RECOPIE

```
class A { ... } ;  
class B : public A { ... } ;  
void fct (B) ; // fct est une fonction recevant un argument de type B  
...  
B b1 (...) ; // arguments éventuels pour un "constructeur usuel"  
fct (b1) ; // appel de fct à qui on doit transmettre b1 par valeur, ce  
//qui implique l'appel d'un constructeur de copie de la classe B
```

Cas1. La classe dérivée (B) n'a pas défini de constructeur de copie

Cas2. La classe dérivée (B) a défini un constructeur de copie

Cas1. La classe dérivée (B) n'a pas défini de constructeur de recopie

- Il y a appel du constructeur de recopie par défaut de B: la recopie se fait membre par membre :
 - La "partie" de b1 appartenant à la classe A sera traitée comme un membre de type A.
 - Il y aura donc appel du constructeur de recopie de A pour tous les membres données correspondants.
- **Si A a défini un tel constructeur, il sera appelé ; dans le cas contraire , on se servira du constructeur de recopie par défaut de A.**

Cas2. La classe dérivée (B) a défini un constructeur de recopie

- le constructeur de recopie de B est appelé

Opérateur d'assignation non hérité

- Si on a un opérateur= dans la classe de base;
 - Si aucun opérateur= dans la classe dérivée;
- ↩ Alors, le compilateur va appeler l'opérateur= de la classe de base et recopie attribut par attribut les attributs de la classe dérivée.

La classe dérivée (B) n'a pas redéfini l'opérateur =

- L'affectation de deux objets de type B se déroule membre à membre en considérant que la "partie héritée de A" constitue un membre.
- les membres propres à B sont traités par l'affectation prévue pour leur type (par défaut ou surdéfinie, suivant le cas).
- La partie héritée de A est traitée par l'affectation prévue dans la classe A, c'est-à-dire par l'opérateur = surdéfini dans A s'il existe, par l'affectation par défaut dans le cas contraire .

La classe dérivée (B) a redéfini l'opérateur =

- Dans ce cas, l'affectation de deux objets de type B fera nécessairement appel à l'opérateur = défini dans B.
- Celui de A ne sera pas appelé, même s'il a été surdéfini.
- **Il faudra donc que l'opérateur = de B prenne en charge tout ce qui concerne l'affectation d'objets de type B, y compris pour ce qui est des membres hérités de A.**

6. Héritage multiple

- Une classe héritière possède plusieurs classes parentes

```
class A :  
{public :  
    int f() {return i;}  
private :  
    int i;  
};
```

```
class B :  
{public :  
    int g() {return 2*i;}  
private :  
    int j;  
};
```

```
class C : public A, public B  
{  
    public :  
        int h();  
}
```

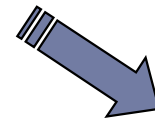
Pb1 :

Que se passe-t-il si un même identifiant est utilisé dans deux classes parentes ?

```
class A :  
{public :  
    int f() {return i;}  
private :  
    int i;  
};
```

```
class B :  
{public :  
    int f() {return 2*i;}  
private :  
    int i;  
};
```

```
class C : public A, public B  
{  
    public :  
        int g();  
};
```



Redéfinir f() dans C

Exemple : pointcoul hérite de point et de coul

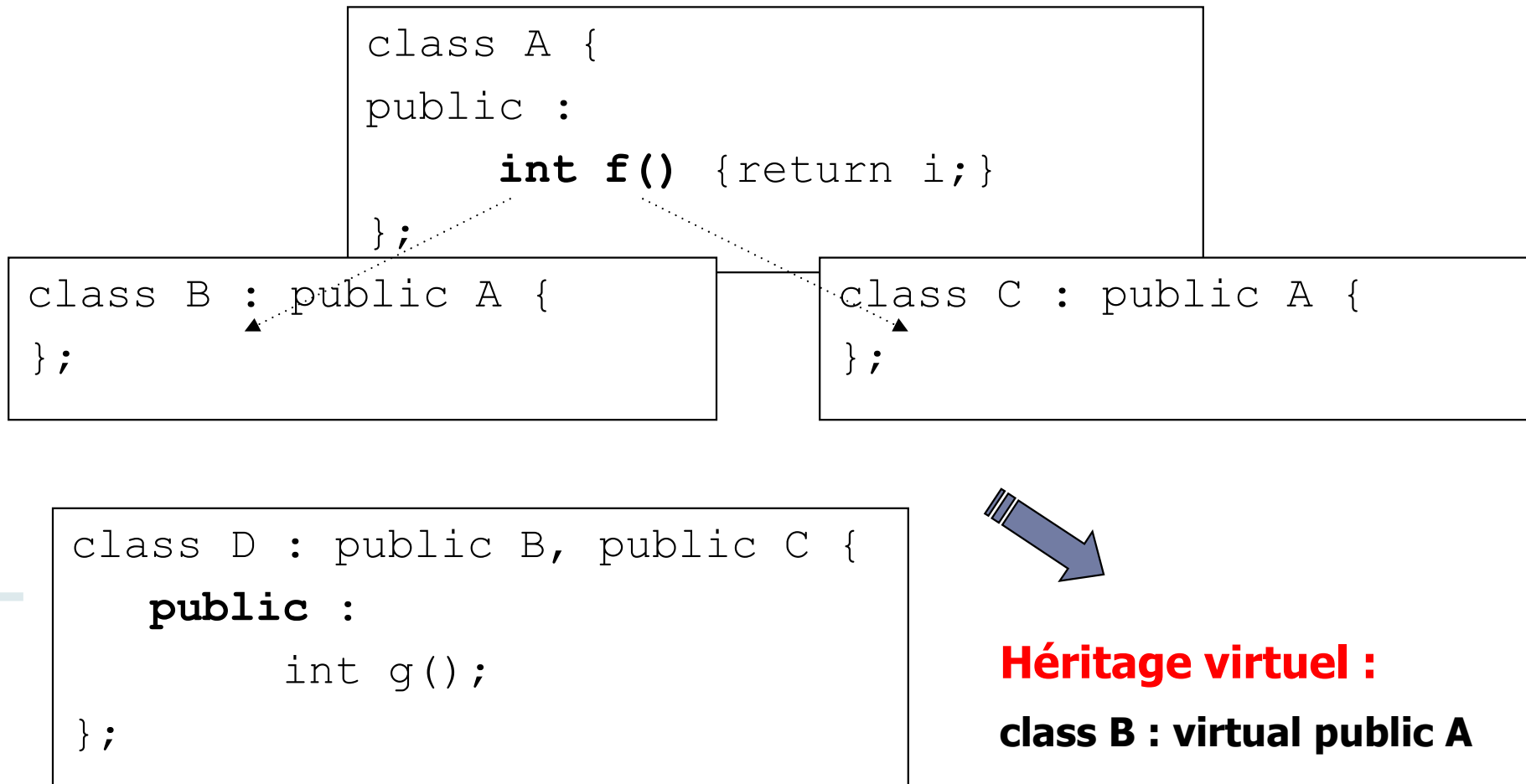
```
class point  
{int x, int y;  
public :  
point (int , int );  
~point ();  
Affiche():  
};
```

```
Class coul  
{ short couleur;  
public :  
coul (int );  
~coul();  
Affiche();  
};
```

```
Class pointcoul : public point, public coul  
{ ...  
public :  
  
pointcoul (int abs, int ord, int cl) : point (abs,  
    ord), coul (cl) { ... }  
~pointcoul ();  
void affiche ()  
{ point::affiche () ; coul::affiche () ;}  
};
```

Pb2 :

Héritage répété : un hydravion hérite des classes d'avion et bateau qui toute deux héritent de véhicule. Que deviennent les caractéristiques de la classe répétée ? Sont-elles dupliquées ou fusionnées ?



Conclusion

- Les classes dérivées sont un mécanisme simple pour définir une nouvelle classe en ajoutant des facilités à une classe existante sans reprogrammer ou recompiler la classe de base.
- En utilisant les classes dérivées d'une classe existante, on définit une interface commune aux classes dérivées de telle manière que les objets de ces classes dérivées sont manipulés de façon identique par certaines parties du programme.



Réutilisation

- On peut ainsi utiliser l'héritage pour les besoins de généralisation, de réutilisation.